

csonus: An open-source collection of opcodes for algorithmic composition and experimental sound design

Valerio Orlandini¹

Conservatorio di Musica “Luigi Cherubini”
valerio.orlandini@stud-phd.consfi.it

Abstract. While the Csound ecosystem provides a vast library of tools for synthesis and signal processing, practitioners in real-time algorithmic composition and experimental sound design often encounter a gap in native, instrument-level opcodes for complex generative processes. This limitation frequently forces composers into hybrid workflows with external environments, introducing latency and synchronization overhead. This paper introduces *csonus*, an open-source collection of C++ opcodes designed to bridge this gap by integrating algorithmic composition and sound generation techniques directly into the Csound instrument block. Built using the Csound Plugin Opcode Framework (CPOF), the package provides efficient implementations of diverse tools, including chaotic systems, cellular automata, Lindenmayer systems, and neural network-driven synthesis. By enabling tightly coupled, multi-level algorithmic decision-making and experimental sound manipulation without external dependencies, *csonus* fosters a more integrated workflow for both studio and live electronics contexts. The library is written in modern C++ and is publicly available under the MIT license.

Keywords: opcodes, algorithmic composition, sound design, open source

1 Introduction

The Csound ecosystem has long been a cornerstone of computer music research and practice, offering a vast and mature library of opcodes for synthesis, signal processing, and spatialization [1,2]. Over the years, the ecosystem has expanded through significant contributions such as the integration of FAUST and Python, and specialized libraries for physical modeling and granular synthesis. In the domain of algorithmic composition [3], CsoundAC has served as a powerful framework for rule-based, stochastic, and data-driven score generation.

However, despite this breadth, practitioners working at the forefront of real-time algorithmic composition and experimental sound design may encounter a gap in the native opcode landscape. While there are add-ons that excel at pre-computing musical structures and generating scores, there is a relative scarcity of native, control-rate or signal-rate opcodes designed for in-instrument algorithmic decision-making. Contemporary compositional practices increasingly demand real-time generative processes executed directly within the instrument block without the latency or overhead of external score parsing.

Consequently, this often leads to bridging Csound with external environments such as Max/MSP, SuperCollider, or Python. While these multi-environment approaches are highly flexible and well-documented, they introduce inter-process communication overhead, synchronization challenges, and workflow fragmentation. For real-time performance and tightly coupled sound-design paradigms, this separation between compositional logic and sound generation can be a significant hindrance.

csonus emerges to address this specific gap. It is a collection of C++ opcodes designed to bring algorithmic composition and experimental synthesis techniques at the Csound instrument level. By leveraging modern C++ and aligning with the ongoing development direction of the Csound core, *csonus* provides efficient implementations of algorithms that are currently underrepresented in the standard distribution.

2 Package Structure

The package *csonus* consists of a series of opcodes oriented towards different aspects of algorithmic composition and experimental sound design, from micro- to macro-structures of a musical piece. To

accomplish this, some opcodes are designed to generate or manipulate audio streams, while others produce modulation signals and complex symbolic sequences to drive and evolve the underlying structures. Finally, there are opcodes that serve as utilities to facilitate the conversion between different data formats. Table 1 lists the currently implemented opcodes with a short description of their functionalities.

Table 1. List of the currently implemented opcodes.

<i>Opcode</i>	<i>Description</i>	<i>Rate</i>
bitinv	Invert selected bits of an 8-bit quantized audio signal	a
bstrtof	Convert a binary string to a normalized float value using a specified bit width	k
bstrtol	Convert a binary string to an integer-style value	k
byteplay	Classic bytebeat waveform generator using user-selected formula and sample-rate reduction	a
ca	A configurable 2D cellular automaton system	k
cheby	Distortion using a Chebyshev polynomial applied to the input signal	a
cryptoverb	Stereo algorithmic reverberation effects with three selectable modes	a
drivers	Different distortion algorithms with a single control parameter	a
genetic	A genetic algorithm that evolves a population of strings towards a target	k
hztomel	Simple frequency to mel conversion opcode	k
linden	A configurable Lindenmayer system generator	k
logistic	Control-rate logistic map generator for chaotic modulation	k
lorenz	Generates three audio-rate signals corresponding to the coordinates of a Lorenz attractor	a
meltohz	Simple mel to frequency conversion opcode	k
neurosc	Neural network-based wavetable oscillator with user-controlled latent weights	a
nowave	Scrambled wavetable oscillator generating deterministic waveforms from a seed and table size	a
perceptron	Simple step-function threshold operator (classical perceptron behavior)	k
phasedist	Phase-distortion oscillator	a
pulsar	Pulsar oscillator (short bursts derived from microsound techniques)	a
quadpan	Quadraphonic panning of a single input signal to four outputs	a
roessler	Generates three audio-rate signals corresponding to the coordinates of a Roessler attractor	a
tent	Control-rate tent map generator for chaotic modulation	k
wavesets	Wavesets-based table manipulation	i

It is modeled after a similar open-source collection of externals for Max/MSP [4], which is developed and maintained by the same author. However, it is not a direct port; rather, it is an adaptation and, in some cases, a complete rewriting of the algorithms to account for the different workflows and data management approaches of Csound compared to Max/MSP.

The code is open-source, released under an MIT license, and is available on GitHub at <https://valeriorlandini.github.io/csonus/>. The library can be built from source using the provided instructions, and binary builds for Linux, Windows and macOS are available as well.

3 Implementation

The opcodes have been written in C++ using the Csound Plugin Opcode Framework (CPOF). The only external dependency is the header-only cross-platform library *SouTeL*¹, featuring DSP routines used in several opcodes and requiring no further dependencies. The code relies on some language features introduced with C++17 (and optionally C++20), but workarounds have been implemented to allow building it with a C++14-compliant compiler as well. Both Csound 6 and 7 are supported and there are no OS-specific constraints, but testing has been carried out mainly on Csound 7 on GNU/Linux.

4 Use Cases

The opcodes of the *csonus* package span diverse functional categories, including oscillators, effects, and structural generators. While these tools can certainly be applied to standard sound design tasks,

¹ <https://github.com/valeriorlandini/soutel>

they are fundamentally united by a specific conceptual focus: multi-level algorithmic composition. Rather than functioning merely as generic building blocks, these objects are explicitly designed to facilitate both the generation of microstructures and their organization into macrostructures.

As outlined in the preceding sections, the core idea is to provide objects that interconnect the different levels of composition. This ensures that these elements are not treated as isolated entities to be assembled only at a later stage, but rather structured into an interconnected network characteristic of rule-based composition.

Therefore, the opcodes of *csonus* can be employed, for example, in:

- Experimental sound design, to generate complex, continuously evolving audio streams as well as to manipulate audio samples using transformation algorithms.
- Algorithmic modulation of other opcodes’ parameters, at both audio and control rates.
- Rule-based sequencing, to produce control structures driven by autonomous systems, applicable to both micro- and macrostructural transformations.
- Sound manipulation in live electronics contexts, utilizing autonomous yet context-sensitive algorithms.

5 Selected Opcodes In-Depth Description

To better exemplify the concept and the implementation details behind the package, some selected opcodes will be described in detail.

5.1 `byteplay`

Bytebeat is a technique for generating raw PCM audio using short, C-style expressions. Evaluated at standard audio sample rates, these formulas rely heavily on bitwise, arithmetic, and logical operators. The concept in these terms was introduced in 2011 by Ville-Matias Heikkilä [5]. A standard bytebeat expression uses a variable, t , which represents the continuously incrementing sample index. In classic 8-bit bytebeat, t cycles from 0 to 255. By utilizing t multiple times and applying various operations, creators can generate complex, evolving audio sequences. `byteplay` introduces users to this method of sound synthesis by allowing them to select from fifteen well-known bytebeat formulas, each generating a distinct audio sequence. An additional parameter enables the reduction of the sample rate at which the waveform is generated. This effectively pitches the audio up or down, which is particularly useful for creating intricate modulation signals.

5.2 `ca`

A cellular automaton [6] is a computational model in which a grid of elements (the *cells*) is arranged across a specific number of dimensions. Each cell has a *state* (typically, in its simplest form, *on* or *off*) and, depending on the states of its neighbours, may maintain its current state or transition to a new one. This type of system has been studied since the 1940s and has found applications in biology, physics, sociology, and other fields [7,8]. `ca` is a configurable, two-state cellular automaton system in which a two-dimensional grid of arbitrary size can be instantiated with custom birth and survival rules. At k-rate, the system evolves, returning an array containing the current states of the cells. The returned array is a one-dimensional flattening of the grid in row-major order.

5.3 `genetic`

Drawing on evolutionary principles and concepts, a genetic algorithm serves as a heuristic method designed to find optimal solutions to complex problems [9]. The algorithm steadily approaches an optimal solution by favoring the fittest individuals (which can be sequences of any kind) through three key operations: crossover, mutation, and selection. The `genetic` opcode applies the aforementioned principles to search for the best string match based on a specific alphabet and mating parameters. The algorithm outputs the current best-fit candidate at k-rate, drawing from an initially randomized population. The output can be mapped to any kind of parameter, allowing for the creation of evolving, algorithmically driven control signals that converge from randomness into structured musical gestures.

5.4 linden

A Lindenmayer system [10] (or L-system) is a type of formal grammar that, starting from an *axiom*, develops it through each iteration according to a series of substitution rules. For example, if the axiom is `AB` and the substitution rules are $A \rightarrow B$ and $B \rightarrow ABA$, subsequent iterations will be `BABA`, `ABABABAB`, and so on. Moreover, an L-system can have *constants*, i.e., items whose rule is the identity, meaning they do not change from one generation to the next. This seemingly simple grammar can generate highly complex and organized structures. Originally formalized to model the development of fungal, bacterial, and plant life, L-systems can be used in music for advanced rule-based sequencing of any parameter. The `linden` opcode allows setting an axiom, the evolution rules, and a maximum sequence length. While this constraint technically deviates from the strict definition of an L-system, it is necessary to avoid the exponential growth of the sequence length, and it is also practical when working with fixed-length patterns. The resulting string, if formatted as a string sequence of zeros and ones, can also be converted into numbers using the included opcodes `bstrtof` and `bstrtoul`, which turn a binary string into floating-point or integer values, respectively.

5.5 neurosc

This opcode implements a wavetable oscillator driven by an autoencoder trained from scratch on a dataset of 15,000 single-cycle waveforms, collected from public domain resources. Once trained in PyTorch, its weights and biases are embedded directly into a C++ header, allowing for native execution without Python dependencies and thus allowing a lower CPU usage. The network’s eight-dimensional latent space is accessible via dedicated opcode parameters, establishing a deterministic mapping between parameter vectors and output waveforms, which are generated by a single-layer decoder. Continuous modulation of these parameters yields smooth spectral and temporal transitions across complex waveforms. While an optional Hann windowing parameter is provided, zero-crossing interpolation at the waveform boundaries is always applied to eliminate discontinuities and associated artifacts.

5.6 wavesets

The `wavesets` opcode implements a series of transformations on an audio buffer (represented as a function table in Csound) by operating directly on its constituent wavesets. Introduced by Trevor Wishart [11], a waveset is defined as an audio segment bounded by a pair of zero-crossings, making it conceptually analogous to a single waveform cycle. This opcode enables various operations on either individual wavesets or groups of consecutive ones. For instance, invoking the *reverse* function with an argument *n* reverses each group of *n* wavesets, while the *stretch* function scales each group by a specified factor. If a transformation alters the overall length of the audio, the output table size is automatically adjusted to accommodate the newly generated buffer.

6 Conclusions

This paper has introduced *csonus*, an open-source collection of C++ opcodes for Csound designed to bridge the gap between algorithmic composition and experimental sound design. By implementing complex generative algorithms, chaotic systems, and advanced synthesis techniques directly at the instrument level, *csonus* eliminates the overhead of hybrid workflows and enables tightly coupled generative processes. The in-depth analysis of selected opcodes illustrates how these tools can be leveraged to create intricate micro- and macro-structural transformations within a unified environment.

Looking ahead, the development of *csonus* will focus on expanding the library’s capabilities, particularly in the realms of machine learning-driven synthesis and automatic sound spatialization. Furthermore, significant effort will be dedicated to enriching the documentation and providing a comprehensive suite of example CSD files to facilitate adoption and lower the learning curve for new users. Ultimately, *csonus* aims to serve as a robust, extensible foundation for computer musicians and researchers, fostering a more integrated and efficient workflow within the broader Csound ecosystem.

References

1. Boulanger, R.: The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing, and Programming. MIT Press (2000)
2. Lazzarini, V., Yi, S., Fitch, J., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: A Sound and Music Computing System. Computer Science, Springer International Publishing (2016)
3. Nierhaus, G.: Algorithmic composition. SpringerWienNewYork, Vienna (2009)
4. Orlandini, V.: sonus, an open-source Max/MSP package for sound experimentation and algorithmic composition (2026)
5. Heikkilä, V.M.: Discovering novel computer music techniques by exploring the space of short computer programs (2011)
6. von Neumann, J.: The Theory of Self-Reproducing Automata. University of Illinois Press, Urbana (1966)
7. Xenakis, I.: Formalized Music: Thought and Mathematics in Composition. Harmonologia series, Pendragon Press (1992)
8. Wolfram, S.: A new kind of science. Wolfram Media, Inc., Champaign (2002)
9. Holland, J.H.: Genetic algorithms. Scientific American **267**(1), 66–73 (1992)
10. Lindenmayer, A.: Mathematical models for cellular interactions in development II. Simple and branching filaments with two-sided inputs. Journal of Theoretical Biology **18**(3), 300–315 (1968)
11. Wishart, T.: Audible design : a plain and easy introduction to practical sound composition. Orpheus the Pantomime (1994)